

Sofiane Alla

Et trois étudiants qui se reconnaîtront et dont je préfère
ne pas divulguer le nom par principe

Rapport Sokoban Progressif



Sommaire

1	Introduction	2
2	Le projet	2
2.1	Sokoban	2
2.1.1	Présentation	2
2.1.2	Histoire	2
2.1.3	Variantes	2
2.2	Objectifs	3
3	Le programme	3
3.1	Les fonctions	3
3.1.1	Déplacement	3
3.1.2	Direction	3
3.1.3	Affichage	5
3.1.4	Génération de niveaux	5
3.1.5	Variables	5
3.1.6	Affichage du numéro du niveau	6
3.2	Assemblage	6
4	Algorithme A*	6
4.1	Introduction général	6
4.2	La classe noeud	7
4.3	La classe etoiles	8
4.4	Création du niveau	9
5	Architecture du projet	9
5.1	Diagramme de Gantt (évolution de notre projet)	9
5.2	Chaînes de traitements des Classes	10
6	Expérimentation et usages	10
6.1	Manuel d'utilisation de notre jeu	10
6.2	Cas d'utilisation	10
6.3	Résultats quantifiables	13
7	Expérience	13
7.1	Organisation	13
7.2	Résolution de problème	14
8	Conclusion	14
8.1	Bilan	14
8.2	Améliorations	14

1 Introduction

Dans ce rapport, nous allons vous présenter le déroulement de la création du jeu Sokoban, ainsi que ses améliorations. Tout d'abord, nous expliquerons le projet en lui-même, c'est à dire les grandes lignes du Sokoban, ainsi que son histoire et les objectifs à atteindre dans ce projet.

Ensuite, nous détaillerons le programme, ses fonctions et la manière dont nous les avons assemblées. Puis nous rentrerons plus en détails dans les termes techniques : on retrouvera notamment l'algorithme A*.

Après cela, nous vous présenterons l'architecture du projet, c'est-à-dire la manière dont les classes interagissent entre-elles. Puis, dans une partie suivante, nous exposerons le cas d'utilisation et les différents résultats quantifiables (temps de calcul du nombre de coups de l'algorithme A*).

Par la suite, nous donnerons notre ressenti sur cette expérience, nous expliquerons comment nous avons résolu certains problèmes, la façon dont nous nous sommes organisés et pour finir nous relèverons les leçons que nous avons pu tirer de ce travail de groupe. Pour conclure, nous ferons un bilan de cette expérience et nous proposerons des améliorations possibles à ce projet.

2 Le projet

2.1 Sokoban

2.1.1 Présentation

Sokoban est un jeu vidéo de puzzle qui consiste à déplacer des caisses sur des cases cibles. Le joueur doit diriger un personnage avec quatre directions possibles : haut, droite, bas et gauche.

Ce jeu est un véritable casse-tête. En effet, il est possible qu'un joueur puisse être bloqué dans un niveau. Ce qui rend le jeu complexe est que le personnage peut uniquement pousser les caisses, donc si cette dernière se retrouve contre un mur, le personnage est dans l'incapacité de la tirer. De plus le personnage ne peut déplacer qu'une caisse à la fois.

2.1.2 Histoire

Sokoban signifie gardien d'entrepôt en français. Ce jeu est originaire du Japon et date du début des années 1980. Il doit sa popularité à des règles très simples mais dont la résolution d'un niveau peut s'avérer être d'une très grande complexité. Initialement, il n'existe qu'une cinquantaine de niveaux mais des milliers d'autres sont maintenant disponibles sur Internet.

2.1.3 Variantes

Il existe plus de vingt variantes du Sokoban. Ces nouveautés sont inspirées de ce jeu de puzzle, chacune d'entre elles possède leurs propres particularités. Quelques fois le design change, les règles ou encore le but. Les variantes de ce jeu affluent en effet du monde entier.

On retrouve par exemple Rescue Rover, originaire des Etats-Unis où les boites permettent de se protéger et de se déplacer dans la map. Il existe aussi Pushy réalisé en Allemagne, avec cette variante la plupart des objets peuvent être déplacés. Enfin, il existe aussi le jeu Kurtan élaboré en Géorgie, il ressemble plus au Sokoban et fait partie des premières variantes. Cependant, dans Kurtan, les espaces de jeu dit "maps" sont plus grands et s'apparentent à différents puzzles liés les uns aux autres, une difficulté s'ajoute donc, en effet, après avoir placé les caisses il faut récupérer une clé afin de sortir de la map et ainsi pouvoir terminer le niveau.

2.2 Objectifs

L'objectif de ce projet est de permettre à un utilisateur de résoudre les niveaux du jeu face à une intelligence artificielle.

Tout d'abord, nous devons réaliser un jeu jouable pour un humain en important plusieurs niveaux de difficulté croissante. Nous ajouterons à cela une interface graphique réalisée à l'aide de Pygame.

Ensuite, nous réaliserons un algorithme A* qui pourra permettre de résoudre automatiquement un niveau afin de faire jouer humain et ordinateur en parallèle au coup-par-coup.

Enfin, la dernière étape consiste à rendre "anytime" l'algorithme de l'intelligence artificielle, c'est-à-dire le rendre capable de résoudre le niveau qu'importe l'endroit et la difficulté, si bien entendu une solution existe car il se pourrait qu'un niveau soit impossible.

3 Le programme

3.1 Les fonctions

3.1.1 Déplacement

La fonction "déplacement" permet d'associer les touches de déplacements : "z", "q", "s" et "d", à des coordonnées sur la map. Ces dernières fonctionnent comme pour un graphique avec x et y. Pour la touche z, notre fonction va associer à x la valeur -1 et à y la valeur 0. A l'inverse, lorsqu'on presse la touche s, on veut faire descendre notre personnage, la fonction donne donc à x la valeur opposé que celle de z, soit 1, en revanche pour y la valeur reste identique. Pour aller vers la droite, on associe à d les coordonnées (0,1). Enfin, pour la touche q (vers la gauche) x prend la valeur 0 et y, -1.

Une autre fonctionnalité se trouve dans cette fonction, elle permet au joueur de recommencer le niveau à tout moment, il suffit juste de presser la touche "e". En effet, ceci consiste à créer une nouvelle grille qui correspond simplement à une copie faite au préalable du niveau avant que le joueur effectue son premier déplacement. Toutes ces coordonnées nous seront utiles pour la fonction suivante.

3.1.2 Direction

Cette fonction permet de contrôler les mouvements du personnage. Le joueur peut déplacer le personnage à l'aide des touches "z" pour monter, "d" pour aller à droite, "s" pour descendre et enfin "q" pour aller à gauche. Cette fonction prend aussi en compte les déplacements des caisses qui sont relatives au déplacement du personnage. En effet, c'est ici que l'on décide si le

personnage est en capacité de pousser un caisse ou non.

Cette fonction peut analyser jusqu'à deux cases après le personnage selon le déplacement souhaité. Il est important de préciser que sur le programme on distingue deux types de personnage : PERSO (un personnage sur une case vide) et PERSO_ON_ZONE (un personnage sur une case destination). Ceci permet d'éviter la confusion entre une case destination recouverte par un personnage et une case destination recouverte par une caisse. En effet, le but du Sokoban est de recouvrir les cases destinations, mais attention, elles doivent l'être par des caisses et non par le personnage. C'est pourquoi il y a deux types de personnage dans le programme mais cela n'influe pas son affichage dans l'interface graphique de Pygame.

Plusieurs cas de figure se présentent alors à nous :

Le joueur se déplace vers une case vide :

Nous allons vérifier si la case qui suit le personnage est vide. Pour effectuer cela nous utilisons les coordonnées associées aux touches de déplacements précédemment définies dans la fonction ci-dessus. Selon le déplacement souhaité, la vérification de la case vide ne sera pas effectuée au même endroit. Après avoir vérifié cela, on va créer de nouvelles coordonnées pour le personnage en le plaçant sur la case qui était vide. Évidemment, la case où il y avait le personnage va devenir une case vide. Si le personnage est sur une case destination (PERSO_ON_ZONE) alors lors du déplacement de ce dernier la case qu'il occupait ne devient pas vide mais devient une case destination.

Le joueur se déplace sur une case destination :

Comme précédemment, nous vérifions si la case qui suit est une case destination selon la touche pressée, et donc ses coordonnées associées. Nous retrouvons ici deux possibilités. Soit le personnage est initialement sur une case vide (PERSO), ce qui implique lors de son déplacement que sa position soit changée en case vide et que la destination devienne un PERSO_ON_ZONE. Soit le personnage est déjà sur une case destination, la seule différence avec le cas précédent est qu'au lieu que la case quittée par le personnage devienne vide, elle devient destination.

Le joueur se déplace avec une caisse :

- La case qui suit la box est vide :

Ici, nous allons juste remplacer des cases par d'autres. En effet, dans ce cas de figure, la case vide va devenir une caisse, la case caisse va devenir un personnage et la case où il y avait le personnage va devenir soit une case vide soit une destination selon le statut du personnage (respectivement PERSO ou PERSO_ON_ZONE).

- La case qui suit la box est une case destination :

Comme vu précédemment, ceci consiste juste à faire du remplacement. La case destination va devenir une BOX_ON_ZONE (ceci permet de progresser dans le niveau), la case de la caisse devient personnage et naturellement la case du personnage devient vide ou destination selon le type de personnage.

- La caisse est déjà sur une case destination :

Pour finir, une subtilité peut s'ajouter aux cas actuels. En effet, si la caisse que l'on déplace est déjà sur une case destination. Ceci explique donc que la case de la caisse (BOX_ON_ZONE) devient un personnage sur une case destination et la case du personnage devient vide ou destination pour les mêmes raisons vues précédemment.

Enfin, si aucune de ces possibilités ne peut être réalisée alors un message d'erreur s'affiche et le personnage ne se déplace pas. Par exemple s'il y a deux caisses l'une derrière l'autre ou un mur devant le personnage, le joueur ne peut pas bouger son personnage.

3.1.3 Affichage

La classe "affichage" permet de créer l'interface graphique du jeu grâce à Pygame. Cette classe permet aussi de gérer la taille de la fenêtre d'affichage de l'écran de jeu. Dans cette classe, les variables sont remplacées par des images. En effet, nous avons un fichier qui contient toutes les variables utiles à la création de ce jeu, c'est ici que chaque caractère est associé à un objet précis (personnage, boîte, mur, destination,...).

3.1.4 Génération de niveaux

Pour la génération de niveau, nous avons créé une fonction 'lecture_fichier' dans le fichier 'niveau.py' capable de transformer un fichier texte (au format .xsb) en une liste de listes représentant le niveau.

Nous tenons à préciser que notre version du jeu comprend 21 niveaux qui nous ont été donnés par notre encadrant de TP M.Yohann JACQUIER.

La fonction 'is_finish' quant à elle est un booléen et permet de vérifier à tout instant si le niveau est fini. Pour cela, nous ne devons plus avoir de cases destination (="DEST"), ainsi que l'absence de case personnage sur une destination (="PERSO_ON_ZONE").

Enfin, la fonction "next_level" permet le passage au niveau suivant si le précédent est terminé. Elle change le numéro du fichier texte à ouvrir. Il est important de préciser que cette fonction ne s'active seulement si "is_finish" renvoie le booléen True.

3.1.5 Variables

Le fichier "variables.py" répertorie les caractères correspondant à chacune des images présentes dans l'interface graphique pygame.

On retrouve le caractère '@' pour le personnage, et le '+' pour le personnage sur une case destination. Le '#' correspond au mur. La case destination est représentée par un point. La boîte est associée au '\$', en revanche si la caisse est sur une zone de rangement elle devient : '*'. Enfin, le vide est remplacé par un espace .

3.1.6 Affichage du numéro du niveau

Nous avons créé le fichier "textniveau.py" afin d'afficher le texte du numéro du niveau actuel. L'affichage du texte se réactualise à chaque changement de niveau (ce qui permet d'optimiser le flux d'information moins conséquent si on réactualise le texte 60 fois par seconde par exemple).

Nous utilisons la classe Niveau dans laquelle est contenu le numéro du niveau en fonction de notre avancé dans le jeu.

De plus, nous adaptons la taille du bloc contenant le texte en fonction du numéro du niveau. En effet, si celui-ci est inférieur à 10, alors on réduit la taille du bloc, car nous avons un seul chiffre. En revanche, s'il est supérieur ou égal à dix, alors on augmente la taille du bloc car le texte comprendra maintenant un nombre à deux chiffres.

3.2 Assemblage

Le fichier "main.py" permet de rassembler les fonctions de déplacements, de génération et de changement de niveau, ainsi qu'appliquer les images de la page d'accueil. Le fichier applique donc l'image de l'arrière plan du jeu ainsi que les différentes images de la page d'accueil.

Tant que rien n'est touché, la page d'accueil reste la même (grâce à un "while" et un booléen "is_playing"). Lorsqu'on appuie sur la touche entrée, "is_playing" prend "True" et le jeu se lance. De plus, ceci met à jour le niveau et lui associe les images correspondantes. Concrètement, on actualise le niveau pour chaque déplacement effectué. Cette fonction permet donc de passer à l'affichage du niveau suivant (si le précédent est réussi).

Enfin, si on appuie sur la croix en haut à droite, le programme s'arrête et la fenêtre du jeu se ferme.

4 Algorithme A*

4.1 Introduction général

Le but de cet algorithme est de créer une Intelligence Artificielle capable de résoudre n'importe quel niveau du jeu que nous avons créé et de manière optimale. Pour cela, on utilise la méthode A* qui est à la "recherche de chemin dans un graphe entre un nœud initial et un nœud final tous deux donnés".

Ainsi en appliquant cette méthode à notre Sokoban, nous devrions créer un algorithme capable de prendre en paramètre la position du personnage ainsi que celles des boîtes et des destinations pour trouver le chemin le plus court pour le personnage afin de remplir toutes les cases destinations avec des boîtes.

Pour être plus concret on calculerai comment aller du personnage à une boîte et d'une boîte à une destination.

Ainsi, notre algorithme A* commence avec un nœud (ici l'étape 0 : représentant le début) et associe à celui-ci à un coût qui représente le nombre de coup(s) déjà joué pour arrivé à ce nœud, de plus il estime la distance séparant le nœud de son point d'arriver que l'on appellera le

coût heuristique. On additionne ensuite le coût et le coût heuristique pour obtenir le coût total et le nœud sera placé dans `open_list`.

On prend en compte le fait que `open_list` représente les nœuds testés pouvant mener à la solution du niveau de manière optimisée. À l'inverse, `closed_list` contient les nœuds ne pouvant pas mener à la solution du niveau, c'est-à-dire si son coût total est supérieur (ou égal) à un autre nœud.

Une fois les "calculs" faits pour un nœud, on les réitère pour la prochaine étape ce qui permet, de proche en proche, de résoudre le niveau car en passant à la prochaine étape, c'est à dire au prochain nœud, on re-calcule les coûts (coûts, coûts heuristique et coûts total) ce qui permettra d'en supprimer en les mettant dans la `closed_list` grâce à la différence des coûts.

4.2 La classe noeud

La classe `Noeud` contient plusieurs fonctionnalités, nous allons vous les exposer ci-dessous.

- Elle calcule le coût théorique : Il représente le nombre de coups nécessaire pour finir le niveau ce qui nous permet d'avoir une estimation du meilleur nombre de coup dans lequel on pourrait déposer une caisse sur une destination(pour vulgariser c'est la distance à vol d'oiseau).
- Elle calcule le coût : Il correspond au nombre de coups déjà joué, il est calculé grâce à la longueur de la liste des parents, c'est-à-dire des coups enregistrés pour arriver à tel ou tel nœud du graphe si on considère bien sûr l'ensemble des possibilités pour résoudre un niveau comme un graphe et un déplacement du personnage comme avancer d'un nœud (grâce à `len(self.parent)-1`).
- Elle calcule le coût heuristique : Ce coup représente l'addition des coups déjà joués ainsi que du coup heuristique.
Ainsi $\text{coûts heuristique} = \text{coups déjà joué} + \text{estimation du nombre de coups restant à jouer}$.
En théorie ce serait donc le nombre de coups à jouer pour pouvoir finir le niveau, ou tout du moins mettre un boîte sur une destination.
- Elle trouve la position du personnage en temps réel sous forme d'un tuple : Ceci grâce à la fonction `"position_perso"`, utile dans le calcul de distance.
- Elle trouve la position des boîtes en temps réel sous forme de tuple dans une liste : Ceci grâce à la fonction `"position_box"` (utile dans le calcul de distance aussi).
- Elle calcule la position des destination en temps réel sous forme de tuple dans une liste : Ceci grâce à la fonction `"position_dest"` (utile dans le calcul de distance également).
- Elle calcule nombre de coups restants entre le personnage et la boîte la plus proche : La fonction `"box_closer"` permet ce calcul.

- Elle crée une liste des distances les plus courtes entre les boîtes et les destinations.
- Elle représente la coût total pour finir le niveau : La fonction "valeur_proba_win" additionne tout les coûts théorique boîte et une destination avec la distance entre un personnage et sa boîte la plus proche. On peut dire que cela représente une estimation du nombre de coûts minimum pour finir le niveau.

4.3 La classe etoiles

La classe Etoiles se compose de plusieurs fonctions. Elles ont toutes un rôle particulier afin de faire fonctionner notre algorithme A*.

- La fonction "start" : se lance lors de l'initialisation de la classe. C'est tout simplement ce qui va permettre de résoudre le puzzle.
- La fonction "grille_str" : sert simplement à afficher proprement une grille dans la console. Elle nous a servie uniquement pour le debug et nous ne l'avons pas appelé dans la version final du projet.
- La fonction "comparer_deux_noeud" : prend deux nœuds en paramètre et compare leur coût heuristique pour ne renvoyer que le nœud ayant le coût total le moins élevé. Les paramètres nœud étant bien entendu des listes comprenant un chemin parcourut par le personnage. Elle n'est plus utilisé car trop lourde mais reste dans le programme.
- La fonction "cout_le_plus_petit" : nous sert à déterminer et renvoyer que le nœud ayant le coût total le plus petit.
- La fonction "nouveau_noeud" : Cette méthode crée à partir d'un nœud déjà existant que l'on appelle parent produit quatre nouveaux nœuds dans les quatre directions. Le noeud parent se retrouve alors dans closed_list (liste comprenant les str créé à partir de tous les " états " des noeuds déjà testés) la liste la plus petite et la retire de open_list (liste comprenant tout les noeuds non-testé).
- La fonction "verif" : est appelé dans nouveau_noeud pour savoir si les nouveaux noeuds créés peuvent être utilisés, c'est à dire si son attribut current_field n'est pas déjà dans un des noeuds de open_list ou closed_list ou si les conditions de la fonction "utilisable" sont bien respectées.
- La fonction "utilisable" : détermine si dans un noeud il y a des boîtes coincées. Si oui le noeud ne sera pas implémenté dans open_list.
- La fonction "dernier_noeud" : elle trouve s'il s'agit du dernier nœud. Concrètement, il s'agit du dernier nœud si il n'y a plus de "DEST" ou de "PERSO_ON_DEST". Nous avons dans un premier temps chercher à déterminer ça avec le coup heuristique mais il y avait des problèmes, cette méthode est donc plus simple et fiable. Tant qu'il n'y a pas de solutions la fonction renvoie None.

- La fonction "retire" : elle prend en paramètre une liste et la supprime, cette fonction aurait pu être utile lorsque deux noeuds ont un " current_field " identique mais nous avons vu le problème autrement, si pour un nouveau noeud le current_field est identique à un autre on ne le met tout simplement pas dans open_lst.
- La fonction "deplacement_IA" : détermine le déplacement que doit effectuer l'IA en fonction de ceux déjà réalisés.
- La fonction "list_coup" : elle renvoie une liste de coup à appliquer au personnage pour résoudre le niveau (exemple : [(0,-1), (1,0),(1,0),(-1,0),...]). Elle est appelée à la fin, si le niveau n'a pas de solution elle renvoie [] c'est à dire une liste vide.

4.4 Création du niveau

La création de la grille pour A* se fait de la même manière que pour la grille de l'humain, seul diffère la manière dont se font les déplacements.

Ainsi si on appuie sur un bouton de déplacement ("z", "s", "d" ou "q"), on fixe un indice que l'on nomme "num_coup" dans la classe Etoile qui commence à 0 et augmente de 1 à chaque fois que l'on joue.

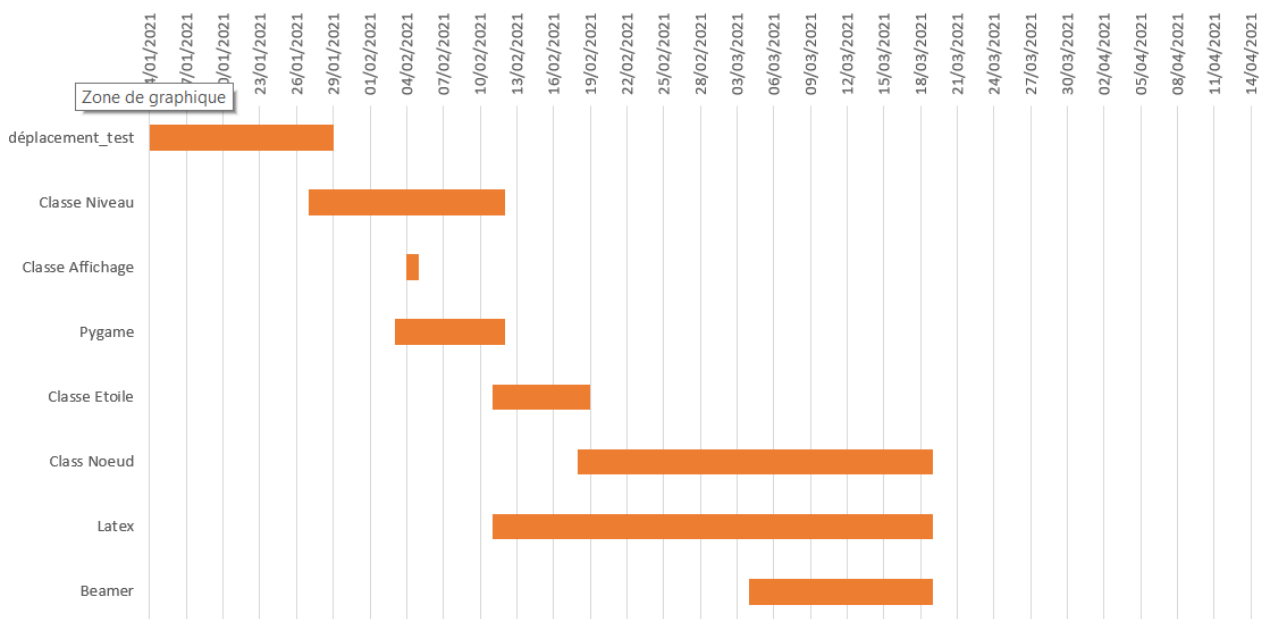
On prend la liste renvoyant les coups à jouer pour finir le niveau pour A* (qui s'appelle "lst_coup"). Ensuite, on se sert du tuple correspondant à l'indice de "num_coup" dans "lst_coup" et on cherche si cela revient à aller à droite, à gauche, en bas ou en haut.

On applique ainsi le changement adéquat de la place du personnage dans la grille de l'IA à la fonction "direction" et on retourne la grille de l'IA qui sera modifier dans "main.py".

5 Architecture du projet

5.1 Diagramme de Gantt (évolution de notre projet)

Pour avoir un aperçu de l'évolution de notre projet, nous avons réalisé un diagramme de Gantt. Celui-ci nous permet de visualiser les tâches que nous avons faites ainsi que leur durée, et de voir dans quel ordre nous les avons faites.



5.2 Chaînes de traitements des Classes

Nous n'avions pas assez de classes pour en faire un diagramme. En revanche, les fichiers interagissent entre-eux, par exemple le fichier "main.py" se sert des classes Niveau (pour le joueur), et le fichier "etoiles.py" interagit avec la classe Noeud (pour l'IA) ainsi que la classe TextNiveau (pour afficher instantanément le niveau où l'intelligence artificielle et le joueur en sont).

6 Expérimentation et usages

6.1 Manuel d'utilisation de notre jeu

Pour pouvoir lancer notre jeu, il faut ouvrir le fichier "main.py". Ensuite il faut appuyer sur la touche Entrée du clavier, et là le jeu se lance.

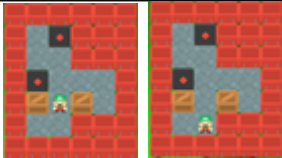
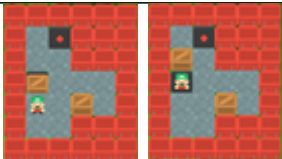
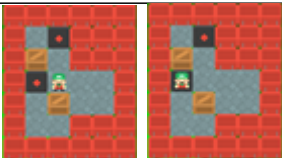
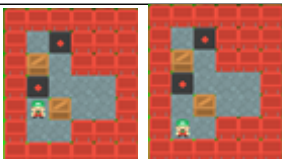
Lest touches pour pouvoir jouer au jeu sont les suivantes :

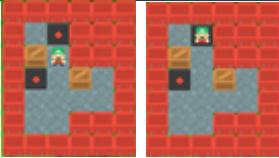
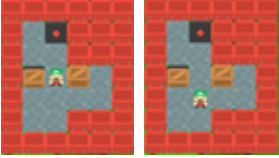
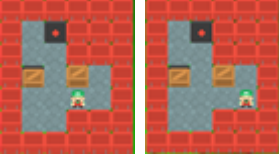
- Pour aller en haut : z
- Pour aller à gauche : q
- Pour aller en bas : s
- Pour aller à droite : d
- Pour revenir au départ quand on est bloqué : e

Si nous voulons arrêter de jouer, il suffit de cliquer sur la croix en haut à droite.

6.2 Cas d'utilisation

Nous allons vous exposer ci-dessous la résolution du premier niveau de notre jeu, à l'aide de capture d'écran de chaque étape de la réalisation.

Explications	Illustrations
Tout d'abord voici le début du niveau (sans aucun mouvements)	
Ensuite on descend deux fois, à l'aide la touche "s"	
Puis on se décale vers la gauche en pressant "q"	
On pousse deux fois la caisse grâce à "z"	
On se décale de trois crans vers la droite (touche "d")	
On va vers la bas	
On pousse la caisse vers la gauche	
On monte	
On se déplace de deux cases vers la gauche	
On descend deux fois	
On se décale sur la droite	

On déplace d'un cran la caisse vers le haut	
On se dirige vers la gauche	
On monte une fois	
On décale la caisse d'une case sur la droite	
On monte deux fois	
On se dirige vers la gauche	
On pousse la caisse d'un cran vers le bas	
On se dirige à droite	
On se déplace de deux cases vers le bas	
On se décale deux fois sur la droite	
On va vers le haut	

On pousse la caisse à gauche	
On va vers le bas	
On se décale d'un cran sur la gauche	
On pousse la caisse vers le haut	

La dernière étape est de pousser la caisse une dernière fois vers le haut et le niveau est terminé, on passe au niveau 2.

6.3 Résultats quantifiables

Nous tenons à préciser que les temps mesurés dans cette section ont tous été fait sur le même ordinateur avec sensiblement les mêmes programmes ouvert et avec un intervalle de temps assez court.

Notre algorithme Etoile a subit plusieurs modifications. Arrivé au niveau 4 la première version mettait un certain temps à résoudre le puzzle (au moins une minute) grâce à l'implémentation de la première version de la fonction "utilisable" de Etoile elle se résolve en quelques secondes.

Le niveau 5 pose aussi quelques problèmes, sans "utilisable" nous avons laissé tourner le programme et au bout de 10 minutes le niveau n'était toujours pas résolu. Avec la première version de la fonction " utilisable " le programme met 3min, cela reste bien mais ce n'est pas suffisant. Nous avons rajouter une fonction "grille str" qui créer un str à partir de la grille, les comparaisons sont donc plus rapide. Pour les premiers niveaux il n'y a pas de différence visible mais pour le niveau 5 on passe de 3min à 2min20. Ensuite nous avons amélioré la fonction "utilisable" ce qui fait que le niveau 5 se résout en 2 min désormais. Cela reste plutôt élevé et même si nous n'avons pas réussi à le rendre plus rapide par manque de temps, la cause du problème vient sûrement du fait qu'il y a beaucoup de calculs inutiles et que python est un langage interprété et que donc il est dans tout les cas, assez lent.

7 Expérience

7.1 Organisation

Ce genre de projet est assez conséquent et demande beaucoup de travail. C'est pourquoi, pour finir à temps, nous avons du nous répartir les tâches afin d'avancer sur différentes choses

simultanément. Etudiant 1 s'est chargé en grande partie du code. A l'inverse, Etudiant 2 s'est plus consacrée à la rédaction du rapport. Enfin, Etudiant 3 et Sofiane ont, eux, rédigés quelques fonctions et participer au rapport.

7.2 Résolution de problème

Nous n'avons pas rencontré de gros problème, en revanche lorsqu'un membre du groupe nécessitait d'aide ses camarades et le professeur étaient là pour l'aider. Lorsqu'on travaille en groupe la valeur fondamentale est l'entraide. En effet, les niveaux de chacun étant différents nous avons tous appris de nouvelles choses. Sans l'assiduité, la présence et l'investissement de chacun, ce projet n'aurait pas pu être réalisé. La communication est quelque chose de primordiale dans ce genre de projet.

8 Conclusion

8.1 Bilan

Le Sokoban du haut de ces 40 ans reste malgré tout très populaire et a connu de nombreuses améliorations au fil des années. Lors de ce projet, nous avons pour objectif de créer ce jeu de A à Z tout en ajoutant des fonctionnalités de résolution, afin que le joueur puisse être débloqué à tout moment. Pour conclure ce rapport, nous allons faire un récapitulatif des fonctionnalités principales de ce jeu. En effet, notre programme contient plus d'une trentaine de fonctions, les plus importantes sont celles qui gèrent le déplacement et le passage de niveaux.

Les fonctions "déplacement" et "direction" sont fondamentales, elles permettent comme nous l'avons vu précédemment de contrôler les mouvements du personnage et des caisses. Lorsque le joueur réussit le niveau, les fonctions "is_finish" et "next_level" vont lui permettre de passer au niveau suivant. Ces fonctionnalités sont certes principales, mais elles ne peuvent fonctionner sans toutes les autres fonctions que contient ce programme.

Nous avons pour objectif de créer un algorithme permettant de résoudre automatiquement un niveau. Cet algorithme A* contient deux classes ("etoiles" et "noeud"), elles vont examiner chaque niveau afin de déterminer le chemin le plus rapide pour résoudre ce dernier. Ces classes permettent de calculer différents coups, qui sont nécessaire à la résolution de chaque niveau, comme nous vous l'avons montré ci-dessus.

Ce projet nous a permis d'obtenir de l'expérience en matière de travaux de groupe. En effet, sans l'entraide, la détermination et l'implication de chacun, cette réalisation n'aurait pas pu avoir lieu.

8.2 Améliorations

Dans un premier temps, lors de la réalisation de notre jeu, nous étions dans l'optique de réaliser un jeu simple. Par la suite nous l'avons amélioré petit à petit. Cependant, d'autres fonctionnalités auraient pu être mises en place. Les améliorations que nous aurions aimés ajouter à notre jeu sont les suivantes :

- Rendre "anytime" l'algorithme de l'intelligence artificielle. En effet, si nous avions eu plus de temps, cette amélioration aurait été la première de nos priorités.
- Faire plusieurs thèmes, c'est-à-dire changer le personnage, les blocs ou les murs par exemple. En effet, le joueur aurait la possibilité de choisir un thème de jeu avant de débiter sa partie.
- Adapter la taille du niveau en fonction de ses dimensions, afin que chaque niveau soit centré dans la fenêtre. Par exemple, le niveau 1 comporte 7 lignes et 7 colonnes contre 16 et 19 pour le niveau 19.
- Avoir trois images de personnage suivant sa direction. Le personnage aurait une image différente selon s'il monte (ou descend), s'il va à gauche ou s'il va à droite.
- Faire une animation, c'est-à-dire une superposition de 3 images en 0.5 sec par exemple pour pouvoir donner un effet de mouvement au personnage.
- Ajouter une touche qui permettrait de revenir au coup précédent. En effet, nous disposons d'une touche qui permet de recommencer le niveau. Mais revenir au début est quelques fois radical, c'est pourquoi, revenir au coup précédent serait idéal (si le niveau est long par exemple).
- Ajouter une élément permettant de pouvoir choisir le niveau que l'on souhaite résoudre. Ceci s'apparenterait à une page de menu, où le joueur sélectionnerait le niveau souhaité à l'aide de sa souris. Il faudrait alors prendre en compte le survol de la souris et l'endroit où elle clique pour pouvoir générer le niveau.
- Génération procédurale (génère aléatoirement un niveau). C'est-à-dire que lorsqu'on lancera notre jeu, un niveau se crée avec des dimensions aléatoires. Le nombre de caisses, de destinations et l'endroit où ils seront placés se fera aussi de manière aléatoire. Malgré tout, l'idéal serait que cette génération aléatoire soit assez réaliste pour que l'on puisse au minimum finir le niveau généré.
- Intégrer dans notre jeu des éléments à la mode « Baba is you ». Baba is you est un jeu de puzzle, dont on peut modifier les règles pendant que l'on joue. Le but du jeu est d'emmener une boule à un endroit d'arrivée (un peu comme Sokoban où l'on doit emmener des caisses sur des destinations).
Lorsqu'on joue, il y a des règles d'écrites dans le niveau, par exemple "WALL IS STOP" on ne peut donc pas passer à travers les murs. Cette règle est en trois blocs : un bloc "WALL", un bloc "IS" et un bloc "STOP". Les trois blocs sont donc par défaut collé et aligné (horizontalement ou verticalement). Il suffit de déplacer un bloc pour briser la chaîne d'alignement et donc la supprimer règle "WALL IS STOP" et par conséquent nous pourrions nous déplacer à travers les murs. Si on rétablit l'alignement, la règle se restaure et on ne peut donc plus passer à travers les murs.
C'est un exemple que nous aurions pu intégrer à notre jeu, qui, bien-sûr, aurait facilité la réalisation de certains niveaux assez complexes.